NEXUS

Complete Backend, AI & Data Implementation Report

Every page. Every role. Every API call. Every database operation. Every data flow.

Google Solution Challenge 2026 — PS 5 Smart Resource Allocation

Stack: React + Firebase + Gemini API + Vertex AI + Google Maps Platform

Environment: All keys stored in .env file — NO GCP Secret Manager

1. Environment Setup — .env Configuration

All API keys, Firebase config, and service credentials are stored in the .env file in the project root. Never commit this file to GitHub — add it to .gitignore immediately.

```
# .env — NEXUS Project Root
# ═══════════════════════════════════════════════════
# FIREBASE CONFIG (from Firebase Console > Project Settings > Web App)
VITE_FIREBASE_API_KEY=your_firebase_api_key
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=123456789
VITE_FIREBASE_APP_ID=1:123456789:web:abcdef
VITE_FIREBASE_MEASUREMENT_ID=G-XXXXXXXXXX
VITE_FIREBASE_DATABASE_URL=https://your-project-default-rtdb.firebaseio.com

# GEMINI API KEY (from aistudio.google.com)
VITE_GEMINI_API_KEY=AIzaSy_your_gemini_api_key

# GOOGLE MAPS (from GCP Console > Credentials)
VITE_GOOGLE_MAPS_API_KEY=AIzaSy_your_maps_api_key

# FIREBASE CLOUD MESSAGING VAPID KEY
VITE_FCM_VAPID_KEY=your_vapid_key_from_firebase_cloud_messaging

# SERVICE ACCOUNT JSON PATH (for local backend scripts only)
GOOGLE_APPLICATION_CREDENTIALS=./service-account.json

# APP CONFIG
VITE_APP_ENV=development
VITE_APP_URL=http://localhost:5173
```

```
// src/lib/firebase.ts — Initialize Firebase with .env variables
import { initializeApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
import { getDatabase } from 'firebase/database';
import { getAuth } from 'firebase/auth';
import { getStorage } from 'firebase/storage';
import { getMessaging } from 'firebase/messaging';

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
  databaseURL: import.meta.env.VITE_FIREBASE_DATABASE_URL,
};
```

```
export const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const rtdb = getDatabase(app);
export const auth = getAuth(app);
export const storage = getStorage(app);
export const messaging = getMessaging(app);

// src/lib/gemini.ts — Gemini client
import { GoogleGenerativeAI } from '@google/generative-ai';
export const genAI = new GoogleGenerativeAI(import.meta.env.VITE_GEMINI_API_KEY);
export const geminiPro = genAI.getGenerativeModel({ model: 'gemini-1.5-pro' });
export const geminiFlash = genAI.getGenerativeModel({ model: 'gemini-1.5-flash'
});
export const geminiVision = genAI.getGenerativeModel({ model: 'gemini-1.5-pro' });
```

2. Complete Firestore Database Schema

All data lives in Firestore. Firebase Realtime Database is used ONLY for volunteer live location and mission status (sub-second updates). Cloud Storage holds images and voice files.

2.1 Firestore Collections

users collection

```
// users/{userId}
{
  id: string,                          // Firebase Auth UID
  name: string,
  email: string,
  phone: string,
  role: 'coordinator'|'fieldworker'|'volunteer',
  ngoId: string,                       // links to ngos collection
  zones: string[],                     // assigned zone IDs
  language: string[],                  // ['kannada','hindi']
  profilePhoto: string|null,           // Cloud Storage URL

  // Volunteer-specific fields
  availability: boolean,               // live toggle — also in RTDB
  travelRadius: number,                // km
  skills: string[],
  emotionalCapacity: 'light'|'moderate'|'intensive',
  avoidCategories: string[],           // private — matching algo only
  impactPoints: number,
  missionsCompleted: number,
  successRate: number,                 // 0-100
  totalHours: number,
  burnoutRisk: 'low'|'medium'|'high',  // Gemini computed weekly
  burnoutScore: number,                // 0-100
  dnaProfile: {
    skillMatch: number, proximity: number,
    emotionalCapacity: number, languageMatch: number,
    pastSuccess: number, availability: number,
    zoneFamiliarity: number, burnoutResistance: number
  },
  badges: string[],
  level: string,
  xp: number,

  // Field worker specific
  primaryLanguage: string,
  offlineZones: string[],              // pre-cached zone IDs

  createdAt: Timestamp,
  updatedAt: Timestamp,
  lastActive: Timestamp
}
```

ngos collection

```
// ngos/{ngoId}
{
  id: string,
  name: string,
  city: string,
  zones: string[],
  needCategories: string[],
  dataChannels: string[],
  trustScore: number,                  // 0-100 Gemini computed
  trustTier: 'gold'|'silver'|'bronze',
  partnerNgoIds: string[],
  collaborationSuggestions: [{
    ngoId: string, ngoName: string,
    reason: string, zones: string[], suggestedAt: Timestamp
  }],
  logoUrl: string|null,
  description: string,
  website: string,
  contactEmail: string,
  createdAt: Timestamp,
  updatedAt: Timestamp
}
```

zones collection

```
// zones/{zoneId}
{
  id: string,
  name: string,                        // 'Hebbal North'
  ward: string,                        // 'Ward 7'
  city: string,
  ngoIds: string[],
  currentScore: number,                // 0-100 live
  riskLevel: 'low'|'medium'|'high'|'critical',
  scoreHistory: [{week:string, score:number, actual:boolean}],
  signalCounts: {food:number, education:number, health:number,
                 substance:number, shelter:number, safety:number},
  activeMissions: number,
  lastIntervention: Timestamp|null,
  forecastScore: number,
  forecastConfidence: number,
  generationalCohort: {
    childrenAtRisk: number, ageGroup: string,
    primaryNeed: string, interventionWindow: string
  },
  safetyProfile: {
    score: number,                     // 0-100
    level: 'safe'|'moderate'|'caution',
    interactions: [{
      date: Timestamp, summary: string,
      sentiment: 'positive'|'neutral'|'negative',
      fieldWorkerId: string
```

```
    }],
    timeOfDayFlags: {morning:string, afternoon:string, evening:string},
    specificFlags: string[]
  },
  geometry: object,                    // GeoJSON polygon
  lat: number, lng: number,            // centroid
  updatedAt: Timestamp
}
```

reports collection

```
// reports/{reportId}
{
  id: string,
  submittedBy: string,                 // userId
  ngoId: string,
  zoneId: string,
  needType: string,
  severity: 'low'|'medium'|'high'|'critical',
  familiesAffected: number,
  location: {lat:number, lng:number, address:string, landmark:string},
  inputType: 'paper_scan'|'voice'|'digital'|'sms'|'community_voice',
  rawData: {
    imageUrl: string|null,             // Cloud Storage
    voiceUrl: string|null,             // Cloud Storage
    transcript: string|null,           // Gemini Audio output
    ocrRaw: string|null,               // Gemini Vision output
    communityPhone: string|null,       // for community voice reports
    referenceNumber: string|null       // community voice ref
  },
  extractedData: object,              // Gemini structured output
  confidence: number,                 // 0-100
  needsReview: boolean,
  safetySignals: string[],            // extracted safety observations
  mergedIntoMissionId: string|null,   // if merged
  status: 'queued'|'processing'|'synced'|'merged'|'flagged',
  syncedAt: Timestamp,
  createdAt: Timestamp
}
```

missions collection

```
// missions/{missionId}
{
  id: string,                          // 'M-047'
  title: string,
  zoneId: string,
  ngoId: string,
  needType: string,
  missionType: 'intervention'|'survey', // NEW — field worker survey type
  priority: 'low'|'high'|'critical',
  description: string,
  targetFamilies: number,
```

```
  assignedTo: string,                  // userId (volunteer or field worker)
  assignedRole: 'volunteer'|'fieldworker',
  dispatchedBy: string,                // coordinatorId
  empathyBrief: string,                // Gemini generated
  empathyLanguage: string,
  empathyDecisionTree: object[],
  safetyBriefing: {                    // NEW — zone safety for volunteer
    score: number, level: string,
    recentNotes: string[], timeRecommendation: string
  },
  resources: [{                        // NEW — pre-assigned resources
    resourceId: string, name: string,
    quantity: number, location: string, distance: number
  }],
  sourceReportIds: string[],           // all merged report IDs
  reportCount: number,                 // how many reports merged
  sourceNgoCount: number,
  status: 'pending'|'dispatched'|'en_route'|'on_ground'|'completed'|'failed',
  needScoreBefore: number,
  needScoreAfter: number|null,
  impactScore: number|null,
  familiesHelped: number|null,
  outcomeNotes: string|null,
  coordinatorRating: number|null,
  isJointMission: boolean,             // inter-NGO collaboration
  jointNgoId: string|null,
  communityVoiceReportIds: string[],   // community reports linked
  createdAt: Timestamp,
  dispatchedAt: Timestamp,
  completedAt: Timestamp|null
  // SUBCOLLECTION: missions/{missionId}/updates
}

// missions/{missionId}/updates/{updateId}
{
  id: string,
  timestamp: Timestamp,
  type: 'status_change'|'voice_update'|'text_update'|'system',
  status: string,
  transcript: string|null,
  text: string|null,
  location: {lat:number, lng:number}|null,
  submittedBy: string
}
```

insights collection

```
// insights/{insightId}
{
  id: string,
  zoneId: string,
  ngoId: string,
  severity: 'watch'|'high'|'critical',
  title: string,
  summary: string,                     // Gemini generated
```

```
  signals: [{label:string, type:string, severity:string}],
  recommendedAction: string,
  sourceReportIds: string[],
  sourceNgoCount: number,
  reportCount: number,
  weeklyTrend: 'rising'|'stable'|'falling',
  patternMatch: string|null,           // historical match
  status: 'active'|'actioned'|'resolved',
  generatedAt: Timestamp,
  resolvedAt: Timestamp|null
}
```

alerts, forecasts, broadcasts, policyBriefs, resources, communityReports

```
// alerts/{alertId}
{ id, zoneId, ngoId, type, severity, title, description, signals,
  weeksToThreshold, confidence, triggeredBy, status, missionIds, createdAt }

// forecasts/{forecastId}
{ id, zoneId, generatedAt,
  predictions:[{week, score, confidence}],
  historicalData:[{week, score, actual}],
  peakWeek, recommendation }

// broadcasts/{broadcastId}
{ id, zoneId, ngoId, week,
  messageKannada, messageHindi, messageEnglish,
  stats:{familiesHelped, pulseImprovement, volunteersDeployed},
  channels, recipientCount, status, scheduledFor, sentAt }

// policyBriefs/{briefId}
{ id, ngoId, period, topNeeds, recommendations,
  contentEnglish, contentHindi, recipientEmail,
  status, sentAt, acknowledgedAt, sdgsAddressed }

// resources/{resourceId}  — NEW
{ id, ngoId, zones:string[], name, category,
  quantity, unit, lowStockThreshold,
  location:{lat, lng, address, contactName, contactPhone},
  distance:number, status:'available'|'low'|'unavailable',
  usageLog:[{missionId, quantityUsed, date, volunteerId}],
  createdAt, updatedAt }

// communityReports/{reportId}  — NEW (Community Voice Channel)
{ id, phone:string, problem:string, problemType:string,
  urgency:'low'|'medium'|'emergency',
  location:{address, lat, lng},
  referenceNumber:string,              // 'NCH-2847'
  status:'received'|'under_review'|'dispatched'|'resolved',
  statusUpdates:[{status, note, timestamp}],
  linkedMissionId:string|null,
  language:string, createdAt:Timestamp }

// notifications/{notificationId}
```

```
{ id, userId, type, title, description, priority,
  read:boolean, actionable:boolean,
  actions:[{label, type, linkedId}],
  linkedId, createdAt }
```

Firebase Realtime Database — Live tracking only

```
// missionTracking/{missionId}
{ volunteerId, status, location:{lat,lng}, lastUpdate, isOnGround }

// volunteerPresence/{volunteerId}
{ available, location:{lat,lng}, lastSeen }
```

3. Authentication Flow — Login, Signup, Role Routing

3.1 Signup Flow

```
// src/pages/auth/SignupPage.tsx
// Step 1: User fills signup form — role, name, email, password, NGO details

async function handleSignup(formData: SignupForm) {
  // 1. Create Firebase Auth user
  const userCredential = await createUserWithEmailAndPassword(
    auth, formData.email, formData.password
  );
  const uid = userCredential.user.uid;

  // 2. Create NGO document if coordinator
  let ngoId = formData.ngoId; // existing NGO join
  if (formData.role === 'coordinator' && formData.createNewNgo) {
    const ngoRef = doc(collection(db, 'ngos'));
    ngoId = ngoRef.id;
    await setDoc(ngoRef, {
      id: ngoId, name: formData.ngoName, city: formData.city,
      zones: [], needCategories: formData.categories,
      dataChannels: formData.channels, trustScore: 0,
      trustTier: 'bronze', partnerNgoIds: [],
      collaborationSuggestions: [], createdAt: serverTimestamp()
    });
  }

  // 3. Create user document in Firestore
  await setDoc(doc(db, 'users', uid), {
    id: uid, name: formData.name, email: formData.email,
    role: formData.role, ngoId,
    zones: formData.role === 'coordinator' ? [] : formData.zones,
    language: [formData.primaryLanguage],
    // volunteer defaults
    availability: false, travelRadius: 5, skills: [],
    emotionalCapacity: 'moderate', avoidCategories: [],
    impactPoints: 0, missionsCompleted: 0, successRate: 0,
    burnoutRisk: 'low', burnoutScore: 0,
    dnaProfile: {skillMatch:50,proximity:50,emotionalCapacity:50,
      languageMatch:50,pastSuccess:50,availability:50,
      zoneFamiliarity:0,burnoutResistance:100},
    badges: [], level: 'Community Helper', xp: 0,
    createdAt: serverTimestamp(), updatedAt: serverTimestamp()
  });

  // 4. Pre-seed city data if new coordinator (Gemini reads public ASHA data)
  if (formData.role === 'coordinator') {
    await preSeedCityData(ngoId, formData.city, formData.zones);
  }

  // 5. Route based on role
  routeByRole(formData.role);
```

```
}
```

3.2 Login Flow + Role-Based Routing

```
// src/pages/auth/LoginPage.tsx
async function handleLogin(email: string, password: string, role: string) {
  const userCredential = await signInWithEmailAndPassword(auth, email, password);
  const uid = userCredential.user.uid;

  // Fetch user document to confirm role
  const userDoc = await getDoc(doc(db, 'users', uid));
  const userData = userDoc.data();

  if (userData.role !== role) {
    throw new Error('Role mismatch — please select the correct role');
  }

  // Store in Zustand auth store
  useAuthStore.setState({ user: userData, ngoId: userData.ngoId });

  // Route to correct dashboard
  if (userData.role === 'coordinator') navigate('/coordinator/dashboard');
  if (userData.role === 'fieldworker') navigate('/fieldworker/dashboard');
  if (userData.role === 'volunteer') navigate('/volunteer/dashboard');
}

// src/store/authStore.ts — Zustand global state
export const useAuthStore = create((set) => ({
  user: null, ngoId: null, loading: true,
  setUser: (user) => set({ user, ngoId: user?.ngoId }),
  clearUser: () => set({ user: null, ngoId: null }),
}));

// src/App.tsx — Auth state listener
onAuthStateChanged(auth, async (firebaseUser) => {
  if (firebaseUser) {
    const snap = await getDoc(doc(db, 'users', firebaseUser.uid));
    useAuthStore.setState({ user: snap.data(), loading: false });
  } else {
    useAuthStore.setState({ user: null, loading: false });
  }
});
```

4. NGO Coordinator — All Pages Backend & Data Flow

4.1 Coordinator Dashboard

```
// src/pages/coordinator/Dashboard.tsx

// DATA FETCHING — on component mount
const { ngoId } = useAuthStore();

// 1. Zone scores for metric cards + map
const [zones, setZones] = useState([]);
useEffect(() => {
  const q = query(collection(db,'zones'),
    where('ngoIds','array-contains', ngoId));
  return onSnapshot(q, snap => {
    setZones(snap.docs.map(d => d.data()));
  }); // LIVE — updates when any zone score changes
}, [ngoId]);

// 2. Latest 2 Gemini insights
const [insights, setInsights] = useState([]);
useEffect(() => {
  const q = query(collection(db,'insights'),
    where('ngoId','==',ngoId),
    where('status','==','active'),
    orderBy('generatedAt','desc'), limit(2));
  return onSnapshot(q, snap => {
    setInsights(snap.docs.map(d => d.data()));
  }); // LIVE — new insight appears automatically
}, [ngoId]);

// 3. Active mission count
const [missionCount, setMissionCount] = useState(0);
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('ngoId','==',ngoId),
    where('status','in',['dispatched','en_route','on_ground']));
  return onSnapshot(q, snap => setMissionCount(snap.size));
}, [ngoId]);

// 4. Available volunteers (Realtime DB)
const [availableCount, setAvailableCount] = useState(0);
useEffect(() => {
  const presenceRef = ref(rtdb, 'volunteerPresence');
  return onValue(presenceRef, snap => {
    let count = 0;
    snap.forEach(child => { if (child.val().available) count++; });
    setAvailableCount(count);
  }); // LIVE — updates on every volunteer toggle
}, []);

// 5. Unread notification count for bell badge
const [unreadCount, setUnreadCount] = useState(0);
```

```
useEffect(() => {
  const q = query(collection(db,'notifications'),
    where('userId','==', auth.currentUser.uid),
    where('read','==',false));
  return onSnapshot(q, snap => setUnreadCount(snap.size));
}, []);
```

4.2 Need Terrain Map

```
// src/pages/coordinator/NeedTerrainMap.tsx

// Google Maps initialization
const mapRef = useRef(null);
const loader = new Loader({ apiKey: import.meta.env.VITE_GOOGLE_MAPS_API_KEY,
  libraries: ['visualization','geometry'] });

// Zones data — LIVE via onSnapshot (same as dashboard)
// When zones state changes → heatmap re-renders
useEffect(() => {
  const map = new google.maps.Map(mapRef.current, {
    center: {lat: 13.0827, lng: 77.5877}, zoom: 13,
    styles: nexusDarkMapStyle // custom dark map style
  });

  // Heatmap layer — weight = zone currentScore / 100
  const heatmapData = zones.map(zone => ({
    location: new google.maps.LatLng(zone.lat, zone.lng),
    weight: zone.currentScore / 100
  }));
  const heatmap = new google.maps.visualization.HeatmapLayer({
    data: heatmapData,
    gradient: ['transparent','#10B981','#F59E0B','#EF4444'],
    radius: 80, opacity: 0.7
  });
  heatmap.setMap(map);

  // Zone click — open slide-over with zone details
  zones.forEach(zone => {
    const polygon = new google.maps.Polygon({
      paths: zone.geometry.coordinates[0].map(([lng,lat]) => ({lat,lng})),
      strokeColor: getRiskColor(zone.riskLevel),
      fillOpacity: 0
    });
    polygon.addListener('click', () => openZoneSlideOver(zone.id));
    polygon.setMap(map);
  });
}, [zones]); // re-renders on live zone updates

// Zone slide-over data fetch
async function openZoneSlideOver(zoneId: string) {
  const zoneDoc = await getDoc(doc(db,'zones',zoneId));
  const recentReports = await getDocs(query(
    collection(db,'reports'),
```

```
    where('zoneId','==',zoneId),
    orderBy('createdAt','desc'), limit(10)));
  setSelectedZone({zone: zoneDoc.data(), reports:
recentReports.docs.map(d=>d.data())});
}
```

4.3 Alerts Feed

```
// src/pages/coordinator/AlertsFeed.tsx

// LIVE alerts — onSnapshot fires when Gemini synthesis creates new alert
const [alerts, setAlerts] = useState([]);
useEffect(() => {
  const q = query(collection(db,'alerts'),
    where('ngoId','==',ngoId),
    where('status','==','active'),
    orderBy('severity'), orderBy('createdAt','desc'));
  return onSnapshot(q, snap => {
    setAlerts(snap.docs.map(d => d.data()));
  });
}, [ngoId]);

// Generate intervention plan — ON DEMAND Gemini call
async function generatePlan(alert: Alert) {
  setGeneratingPlan(alert.id);
  const zoneData = await getDoc(doc(db,'zones',alert.zoneId));
  const prompt = `You are a community intervention specialist.
    Zone: ${alert.zoneId}. Alert: ${alert.title}.
    Signals: ${JSON.stringify(alert.signals)}.
    Zone history: ${JSON.stringify(zoneData.data()?.scoreHistory)}.
    Generate a specific 3-step intervention plan with volunteer requirements,
    resource needs, and expected timeline. Return as JSON:`,
    {steps[], volunteerCount, resourcesNeeded[], estimatedDays, urgency}`;

  const result = await geminiPro.generateContent(prompt);
  const plan = JSON.parse(result.response.text());
  setPlan({ alertId: alert.id, ...plan });
  setGeneratingPlan(null);
}
```

4.4 Volunteer Management

```
// src/pages/coordinator/VolunteerManagement.tsx

// Fetch all volunteers in NGO's zones
const [volunteers, setVolunteers] = useState([]);
useEffect(() => {
  const q = query(collection(db,'users'),
    where('role','==','volunteer'),
    where('ngoId','==',ngoId));
  const unsub = onSnapshot(q, snap => {
```

```
    setVolunteers(snap.docs.map(d => d.data()));
  });
  return unsub;
}, [ngoId]);

// Live availability from Realtime DB
useEffect(() => {
  return onValue(ref(rtdb,'volunteerPresence'), snap => {
    const presence = {};
    snap.forEach(child => { presence[child.key] = child.val(); });
    setPresence(presence); // merge into volunteer cards
  });
}, []);

// AI Match scoring for a specific mission — ON DEMAND
async function scoreVolunteersForMission(mission: Mission) {
  const availableVols = volunteers.filter(v =>
    presence[v.id]?.available && !v.avoidCategories.includes(mission.needType));

  const prompt = `Score these volunteers for mission: ${JSON.stringify(mission)}.
    Volunteers: ${JSON.stringify(availableVols.map(v => ({
      id:v.id, name:v.name, skills:v.skills, dnaProfile:v.dnaProfile,
      location:presence[v.id]?.location, language:v.language,
      successRate:v.successRate, burnoutRisk:v.burnoutRisk
    }))))}.
    Score each on: skillMatch, proximity, languageMatch, emotionalCapacity,
    pastSuccess, zoneFamiliarity, availability, burnoutResistance (0-100 each).
    Return JSON: [{volunteerId, overallScore, dimensions:{...}, reasoning:string}]
    sorted by overallScore descending.`;

  const result = await geminiPro.generateContent(prompt);
  return JSON.parse(result.response.text());
}
```

4.5 Missions Page — Create, Track, Merge

```
// src/pages/coordinator/Missions.tsx

// LIVE mission list
const [missions, setMissions] = useState([]);
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('ngoId','==',ngoId),
    orderBy('status'), orderBy('createdAt','desc'));
  return onSnapshot(q, snap => setMissions(snap.docs.map(d=>d.data())));
}, [ngoId]);

// LIVE mission tracking via Realtime DB
function trackMission(missionId: string) {
  return onValue(ref(rtdb, `missionTracking/${missionId}`), snap => {
    setTrackingData(prev => ({...prev, [missionId]: snap.val()}));
  });
}
```

```
// DUPLICATE DETECTION — check before creating mission
async function checkDuplicate(zoneId: string, needType: string) {
  const q = query(collection(db,'missions'),
    where('zoneId','==',zoneId),
    where('needType','==',needType),
    where('status','in',['pending','dispatched','en_route','on_ground']));
  const snap = await getDocs(q);
  return snap.empty ? null : snap.docs[0].data(); // returns existing mission
}

// CREATE MISSION — with duplicate check
async function createMission(formData: MissionForm) {
  const existing = await checkDuplicate(formData.zoneId, formData.needType);
  if (existing) {
    setDuplicateWarning(existing); // show warning UI
    return;
  }

  // Generate empathy brief + safety briefing via Gemini
  const zoneDoc = await getDoc(doc(db,'zones',formData.zoneId));
  const zone = zoneDoc.data();
  const volunteer = volunteers.find(v => v.id === formData.assignedTo);

  const briefPrompt = `Generate a volunteer mission brief in
${volunteer.language[0]}.
    Mission: ${formData.needType} in ${zone.name}.
    Zone safety: ${JSON.stringify(zone.safetyProfile)}.
    Community context: ${JSON.stringify(zone.signalCounts)}.
    Return JSON: {openingLine, avoidances:string[], decisionTree:[{if,then}],
    safetyBriefing:{score, level, recentNotes, timeRecommendation}}`;

  const briefResult = await geminiPro.generateContent(briefPrompt);
  const brief = JSON.parse(briefResult.response.text());

  // Fetch nearby resources
  const resourcesSnap = await getDocs(query(
    collection(db,'resources'),
    where('zones','array-contains',formData.zoneId),
    where('status','!=','unavailable')));
  const resources = resourcesSnap.docs.map(d=>d.data());

  // Create mission document
  const missionRef = doc(collection(db,'missions'));
  const zone_score = zone.currentScore;
  await setDoc(missionRef, {
    id: missionRef.id,
    ...formData,
    empathyBrief: brief.openingLine,
    empathyLanguage: volunteer.language[0],
    empathyDecisionTree: brief.decisionTree,
    safetyBriefing: brief.safetyBriefing,
    resources: resources.slice(0,4),
    sourceReportIds: formData.reportIds || [],
    reportCount: formData.reportIds?.length || 0,
    needScoreBefore: zone_score,
```

```
    status: 'pending',
    isJointMission: false,
    createdAt: serverTimestamp()
  });

  // Notify volunteer via FCM
  await createNotification({
    userId: formData.assignedTo,
    type: 'mission_dispatch',
    title: `New mission — ${zone.name} ${formData.needType}`,
    description: brief.openingLine,
    linkedId: missionRef.id,
    actionable: true,
    actions: [{label:'Accept',type:'primary'},{label:'Decline',type:'danger'}]
  });
}

// MERGE REPORT INTO EXISTING MISSION
async function mergeReportIntoMission(reportId: string, missionId: string) {
  const batch = writeBatch(db);
  batch.update(doc(db,'missions',missionId), {
    sourceReportIds: arrayUnion(reportId),
    reportCount: increment(1)
  });
  batch.update(doc(db,'reports',reportId), {
    mergedIntoMissionId: missionId,
    status: 'merged'
  });
  await batch.commit();
  // Notify coordinator of merge
  await createNotification({userId: coordinatorId,
    type:'mission_updated', title:`New report merged into ${missionId}`});
}
```

4.6 Intelligence Hub — Merged Page (Gemini Insights + Forecast + Trust Fabric + Living Constitution + Community Echo)

```
// src/pages/coordinator/IntelligenceHub.tsx
// Tabs: Gemini Insights | Forecast | Trust & Impact | Living Constitution |
Community Echo

// ─── TAB 1: GEMINI INSIGHTS ───
// Fetch synthesized insights — LIVE
const insightsQuery = query(collection(db,'insights'),
  where('ngoId','==',ngoId), where('status','==','active'),
  orderBy('severity'), orderBy('generatedAt','desc'));
onSnapshot(insightsQuery, snap => setInsights(snap.docs.map(d=>d.data())));

// SYNTHESIS — run Gemini on all zone reports
async function runSynthesis(zoneId?: string) {
  const reportsSnap = await getDocs(query(collection(db,'reports'),
    zoneId ? where('zoneId','==',zoneId) : where('ngoId','==',ngoId),
    where('status','==','synced'),
```

```
    orderBy('createdAt','desc'), limit(100)));
  const reports = reportsSnap.docs.map(d=>d.data());

  const prompt = `Analyze ${reports.length} community field reports.
    Reports: ${JSON.stringify(reports.map(r=>({
      needType:r.needType, severity:r.severity,
      familiesAffected:r.familiesAffected, zone:r.zoneId,
      safetySignals:r.safetySignals
    }))))}.
    Identify top 5 converging signals across zones.
    For each: check if pattern matches historical crisis signatures.
    Return JSON array: [{zoneId, title, summary, signals, severity,
    recommendedAction, patternMatch, weeklyTrend, reportCount}]`;

  const result = await geminiFlash.generateContent(prompt);
  const synthesized = JSON.parse(result.response.text());

  // Save each insight to Firestore
  for (const insight of synthesized) {
    await addDoc(collection(db,'insights'), {
      ...insight, ngoId, status:'active',
      generatedAt: serverTimestamp()
    });
    // Update zone signal counts
    await updateDoc(doc(db,'zones',insight.zoneId), {
      [`signalCounts.${insight.signals[0]?.type}`]: increment(1),
      updatedAt: serverTimestamp()
    });
  }
}

// ─── TAB 2: FORECAST ───
// Fetch forecasts for all zones
const forecastsSnap = await getDocs(query(collection(db,'forecasts'),
  where('zoneId','in', ngoZoneIds),
  orderBy('generatedAt','desc')));

// Generate forecast using Gemini (Vertex AI substitute for hackathon)
async function generateForecast(zoneId: string) {
  const zone = await getDoc(doc(db,'zones',zoneId));
  const history = zone.data()?.scoreHistory || [];

  const prompt = `Analyze this zone's 8-week need score history:
${JSON.stringify(history)}.
    Predict next 4 weeks need scores (0-100) with confidence %.
    Identify seasonal patterns and peak week.
    Return JSON: {predictions:[{week,score,confidence}],
    peakWeek, recommendation, trend}`;

  const result = await geminiPro.generateContent(prompt);
  const forecast = JSON.parse(result.response.text());

  const forecastRef = doc(collection(db,'forecasts'));
  await setDoc(forecastRef, {
    ...forecast, zoneId, historicalData: history,
```

```
    generatedAt: serverTimestamp()
  });
  // Update zone forecast fields
  await updateDoc(doc(db,'zones',zoneId), {
    forecastScore: forecast.predictions[0]?.score,
    forecastConfidence: forecast.predictions[0]?.confidence
  });
}

// ─── TAB 3: TRUST & IMPACT ───
// 3A: Impact Overview sub-tab
// Fetch completed missions with outcomes for charts
const completedMissions = await getDocs(query(collection(db,'missions'),
  where('ngoId','==',ngoId), where('status','==','completed'),
  where('needScoreAfter','!=',null), orderBy('completedAt','desc')));

// Compute metrics from data (no AI needed — pure aggregation)
const totalFamilies = completedMissions.reduce((sum,m)=>sum+m.familiesHelped,0);
const avgReduction = completedMissions.reduce((sum,m)=>
  sum+(m.needScoreBefore-m.needScoreAfter),0) / completedMissions.length;

// 3B: Trust Ledger sub-tab
// NGO trust score from ngos collection — LIVE
const ngoDoc = onSnapshot(doc(db,'ngos',ngoId), snap => setNgo(snap.data()));

// Recompute trust score via Gemini
async function computeTrustScore() {
  const missions = completedMissions; // from above
  const prompt = `Compute an NGO trust score (0-100) based on:
    Mission outcomes: ${JSON.stringify(missions.map(m=>({
      needScoreBefore:m.needScoreBefore, needScoreAfter:m.needScoreAfter,
      familiesHelped:m.familiesHelped, rating:m.coordinatorRating
    }))))}.
    Score dimensions: missionSuccessRate, needScoreReduction,
    volunteerRetention, reportAccuracy, communityFeedback.
    Return JSON: {overall, dimensions:{...}, tier:'gold'|'silver'|'bronze'}`;

  const result = await geminiFlash.generateContent(prompt);
  const trust = JSON.parse(result.response.text());
  await updateDoc(doc(db,'ngos',ngoId), {
    trustScore: trust.overall, trustTier: trust.tier,
    updatedAt: serverTimestamp()
  });
}

// 3C: Donor Report sub-tab — generate PDF
async function generateDonorReport(period: string, sections: string[]) {
  const prompt = `Generate a professional NGO donor impact report for ${period}.
    Data: families helped=${totalFamilies}, avg reduction=${avgReduction}%,
    missions=${completedMissions.length}, trust score=${ngo.trustScore}.
    Mission details: ${JSON.stringify(completedMissions.slice(0,20))}.
    Write in formal English. Include executive summary, impact metrics,
    zone highlights, volunteer achievements, and SDG alignment.
    Return as structured JSON: {executiveSummary, sections:[{title,content}]}`;

  const result = await geminiPro.generateContent(prompt);
```

```
  const report = JSON.parse(result.response.text());
  // Frontend renders as PDF using jsPDF
  generatePDF(report);
}

// ─── TAB 4: LIVING CONSTITUTION ───
// Fetch existing briefs
const briefs = await getDocs(query(collection(db,'policyBriefs'),
  where('ngoId','==',ngoId), orderBy('createdAt','desc')));

async function generatePolicyBrief(period: string) {
  // Collect all zone data
  const allZones = zones; // from live state
  const topNeeds = allZones.flatMap(z =>

Object.entries(z.signalCounts).map(([type,count])=>({zone:z.name,type,count})))
    .sort((a,b)=>b.count-a.count).slice(0,10);

  const prompt = `Generate a formal Indian government policy brief for ${period}.
    Top community needs: ${JSON.stringify(topNeeds)}.
    Write in government document format with:
    1. Executive Summary 2. Top 10 Unmet Needs (numbered, with zone, families)
    3. Data Evidence 4. Recommended Government Actions 5. SDG Alignment.
    Also generate Hindi translation.
    Return JSON: {contentEnglish, contentHindi,
    topNeeds:[{rank,need,zone,families,trend}],
    recommendations:string[], sdgsAddressed:string[]}`;

  const result = await geminiPro.generateContent(prompt);
  const brief = JSON.parse(result.response.text());

  const briefRef = doc(collection(db,'policyBriefs'));
  await setDoc(briefRef, {
    ...brief, ngoId, period,
    status: 'draft', createdAt: serverTimestamp()
  });
}

async function sendToGovernment(briefId: string, recipientEmail: string) {
  await updateDoc(doc(db,'policyBriefs',briefId), {
    status: 'sent', recipientEmail, sentAt: serverTimestamp()
  });
  // In production: email via SendGrid / Firebase Extension
  // For hackathon: download PDF + manual send
}

// ─── TAB 5: COMMUNITY ECHO ───
const broadcasts = await getDocs(query(collection(db,'broadcasts'),
  where('ngoId','==',ngoId), orderBy('createdAt','desc'), limit(12)));

async function generateEcho(zoneId: string, language: string) {
  const zone = zones.find(z=>z.id===zoneId);
  const recentMissions = await getDocs(query(collection(db,'missions'),
    where('zoneId','==',zoneId), where('status','==','completed'),
    orderBy('completedAt','desc'), limit(5)));
```

```
  const missionsData = recentMissions.docs.map(d=>d.data());

  const prompt = `Write a warm community broadcast in ${language} for
${zone.name}.
    This week: ${missionsData.length} missions completed, families helped this
week,
    community pulse score changed from ${zone.scoreHistory.slice(-2)[0]?.score}
    to ${zone.currentScore}.
    Mission types: ${missionsData.map(m=>m.needType).join(', ')}.
    Keep it warm, specific, 3-4 sentences. Community members should feel heard.
    Also provide English and Hindi versions.
    Return JSON: {messageKannada, messageHindi, messageEnglish}`;

  const result = await geminiFlash.generateContent(prompt);
  const echo = JSON.parse(result.response.text());

  const totalFamilies = missionsData.reduce((s,m)=>s+m.familiesHelped,0);
  await addDoc(collection(db,'broadcasts'), {
    ...echo, zoneId, ngoId,
    stats:{ familiesHelped: totalFamilies,
      pulseImprovement: zone.currentScore - (zone.scoreHistory.slice(-
2)[0]?.score||0),
      volunteersDeployed: missionsData.length },
    status:'draft', createdAt: serverTimestamp()
  });
}
```

4.7 Resource Inventory Page

```
// src/pages/coordinator/ResourceInventory.tsx

// Fetch resources for NGO zones
const [resources, setResources] = useState([]);
useEffect(() => {
  const q = query(collection(db,'resources'),
    where('ngoId','==',ngoId));
  return onSnapshot(q, snap => setResources(snap.docs.map(d=>d.data())));
  // LIVE — updates when any resource is added or quantity changes
}, [ngoId]);

// Add new resource
async function addResource(formData: ResourceForm) {
  const resourceRef = doc(collection(db,'resources'));
  await setDoc(resourceRef, {
    ...formData, id: resourceRef.id, ngoId,
    status: 'available', usageLog: [],
    createdAt: serverTimestamp(), updatedAt: serverTimestamp()
  });
}

// Log usage — decrements quantity
async function logUsage(resourceId: string, quantity: number,
                        missionId: string, volunteerId: string) {
```

```
  const batch = writeBatch(db);
  const resourceRef = doc(db,'resources',resourceId);
  const resource = resources.find(r=>r.id===resourceId);
  const newQty = resource.quantity - quantity;

  batch.update(resourceRef, {
    quantity: newQty,
    status: newQty <= resource.lowStockThreshold ? 'low' : 'available',
    usageLog: arrayUnion({missionId, quantityUsed:quantity,
      volunteerId, date: serverTimestamp()}),
    updatedAt: serverTimestamp()
  });
  // Update mission with resource used
  batch.update(doc(db,'missions',missionId), {
    resources: arrayUnion({resourceId, quantityUsed: quantity})
  });
  await batch.commit();

  // Notify coordinator if low stock
  if (newQty <= resource.lowStockThreshold) {
    await createNotification({
      userId: auth.currentUser.uid,
      type: 'system',
      title: `Low stock: ${resource.name}`,
      description: `Only ${newQty} ${resource.unit} remaining at
${resource.location.address}`
    });
  }
}
```

5. Field Worker — All Pages Backend & Data Flow

5.1 Field Worker Dashboard

```
// src/pages/fieldworker/Dashboard.tsx

// Recent reports — LIVE
useEffect(() => {
  const q = query(collection(db,'reports'),
    where('submittedBy','==',userId),
    orderBy('createdAt','desc'), limit(5));
  return onSnapshot(q, snap => setReports(snap.docs.map(d=>d.data())));
}, [userId]);

// Active mission assigned to field worker
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('assignedTo','==',userId),
    where('assignedRole','==','fieldworker'),
    where('status','in',['dispatched','en_route','on_ground']));
  return onSnapshot(q, snap => {
    setActiveMission(snap.empty ? null : snap.docs[0].data());
  });
}, [userId]);

// Offline queue count from IndexedDB
useEffect(() => {
  getOfflineQueueCount().then(setQueueCount);
}, []);
```

5.2 Scan Survey — Gemini Vision OCR

```
// src/pages/fieldworker/ScanSurvey.tsx

async function processScanUpload(imageFile: File) {
  setProcessing(true);

  // Convert image to base64 for Gemini Vision
  const base64 = await fileToBase64(imageFile);

  // Call Gemini Vision API directly from frontend
  const imagePart = {
    inlineData: { data: base64, mimeType: imageFile.type }
  };
  const prompt = `Extract all data from this community survey form.
    Return JSON: {needType, severity:'low'|'medium'|'high'|'critical',
    familiesAffected:number, location:string, landmark:string,
    additionalNotes:string, confidence:number (0-100 overall),
    fieldConfidences:{needType:number, severity:number, families:number}}`;
```

```
  try {
    const result = await geminiVision.generateContent([prompt, imagePart]);
    const extracted = JSON.parse(result.response.text());
    setExtractedData(extracted);

    // Upload image to Cloud Storage
    const imageRef = storageRef(storage, `surveys/${userId}/${Date.now()}.jpg`);
    await uploadBytes(imageRef, imageFile);
    const imageUrl = await getDownloadURL(imageRef);
    setImageUrl(imageUrl);
  } catch (err) {
    setError('Could not read this image — please try again');
  }
  setProcessing(false);
}

async function submitReport(extractedData: any, imageUrl: string, location: any) {
  const report = {
    submittedBy: userId, ngoId, zoneId: selectedZone,
    needType: extractedData.needType,
    severity: extractedData.severity,
    familiesAffected: extractedData.familiesAffected,
    location, inputType: 'paper_scan',
    rawData: { imageUrl, voiceUrl:null, transcript:null,
      ocrRaw: JSON.stringify(extractedData) },
    extractedData, confidence: extractedData.confidence,
    needsReview: extractedData.confidence < 80,
    safetySignals: extractedData.safetySignals || [],
    status: 'processing',
    createdAt: serverTimestamp()
  };

  // CHECK FOR EXISTING MISSION — duplicate detection
  const existingMission = await checkDuplicate(selectedZone,
extractedData.needType);

  if (navigator.onLine) {
    const reportRef = await addDoc(collection(db,'reports'), report);

    if (existingMission) {
      // MERGE into existing mission
      await mergeReportIntoMission(reportRef.id, existingMission.id);
      setSubmitResult({ merged: true, missionId: existingMission.id });
    } else {
      await updateDoc(reportRef, { status: 'synced' });
      setSubmitResult({ merged: false });
      // Trigger synthesis if zone has >= 5 new reports
      await triggerSynthesisIfNeeded(selectedZone, ngoId);
    }
  } else {
    // OFFLINE — save to IndexedDB
    await saveToIndexedDB({ ...report, imageUrl, id: generateLocalId() });
    setQueueCount(prev => prev + 1);
  }
}
```

```
// Auto-sync when back online
window.addEventListener('online', async () => {
  const queued = await getAllFromIndexedDB();
  for (const item of queued) {
    const reportRef = await addDoc(collection(db,'reports'),
      {...item, status:'synced', syncedAt: serverTimestamp()});
    const existing = await checkDuplicate(item.zoneId, item.needType);
    if (existing) await mergeReportIntoMission(reportRef.id, existing.id);
    await deleteFromIndexedDB(item.id);
  }
  setQueueCount(0);
});
```

5.3 Voice Report — Gemini Audio

```
// src/pages/fieldworker/VoiceReport.tsx

// Record audio using MediaRecorder API
const [recording, setRecording] = useState(false);
const mediaRecorder = useRef(null);
const audioChunks = useRef([]);

function startRecording(language: string) {
  navigator.mediaDevices.getUserMedia({ audio: true }).then(stream => {
    mediaRecorder.current = new MediaRecorder(stream);
    audioChunks.current = [];
    mediaRecorder.current.ondataavailable = e => audioChunks.current.push(e.data);
    mediaRecorder.current.start();
    setRecording(true);
  });
}

async function stopRecordingAndProcess() {
  mediaRecorder.current.stop();
  setRecording(false);
  setProcessing(true);

  const audioBlob = new Blob(audioChunks.current, { type: 'audio/wav' });

  // Upload audio to Cloud Storage
  const audioRef = storageRef(storage, `voice/${userId}/${Date.now()}.wav`);
  await uploadBytes(audioRef, audioBlob);
  const voiceUrl = await getDownloadURL(audioRef);

  // Convert audio to base64 for Gemini Audio
  const base64Audio = await blobToBase64(audioBlob);

  const audioPart = {
    inlineData: { data: base64Audio, mimeType: 'audio/wav' }
  };
  const prompt = `Transcribe this voice report in ${selectedLanguage}.
    Then extract community data.
    Return JSON: {transcript:string (original language),
```

```
    transcriptEnglish:string (English translation),
    needType:string, severity:string,
    familiesAffected:number, location:string,
    safetySignals:string[], confidence:number}`;

  const result = await geminiVision.generateContent([prompt, audioPart]);
  const extracted = JSON.parse(result.response.text());
  setTranscript(extracted.transcript);
  setExtractedData(extracted);
  setVoiceUrl(voiceUrl);
  setProcessing(false);
}
```

5.4 Active Mission — Field Worker Survey Mission

```
// src/pages/fieldworker/ActiveMission.tsx
// Field workers receive SURVEY missions — coordinator-assigned fieldwork

// Fetch active survey mission
const [mission, setMission] = useState(null);
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('assignedTo','==',userId),
    where('missionType','==','survey'),
    where('status','in',['dispatched','en_route','on_ground']));
  return onSnapshot(q, snap => {
    setMission(snap.empty ? null : snap.docs[0].data());
  });
}, [userId]);

// Update status — written to RTDB for sub-second coordinator visibility
async function updateStatus(status: string) {
  // 1. Realtime DB — instant update
  await set(ref(rtdb, `missionTracking/${mission.id}`), {
    volunteerId: userId, status,
    location: await getCurrentLocation(),
    lastUpdate: serverTimestamp(), isOnGround: status===='on_ground'
  });
  // 2. Firestore — permanent record
  await addDoc(collection(db,'missions',mission.id,'updates'), {
    type: 'status_change', status,
    timestamp: serverTimestamp(), submittedBy: userId
  });
}

// Submit voice field update
async function submitVoiceUpdate(audioBlob: Blob) {
  const base64 = await blobToBase64(audioBlob);
  const audioPart = { inlineData:{data:base64, mimeType:'audio/wav'} };
  const result = await geminiVision.generateContent(
    ['Transcribe this field update. Extract key findings. Return JSON:' +
     '{transcript, keyFindings:string[], familiesVisited:number}', audioPart]);
  const update = JSON.parse(result.response.text());
```

```
  await addDoc(collection(db,'missions',mission.id,'updates'), {
    type:'voice_update', transcript:update.transcript,
    timestamp:serverTimestamp(), submittedBy:userId
  });
}
```

6. Volunteer — All Pages Backend & Data Flow

6.1 Volunteer Dashboard

```
// src/pages/volunteer/Dashboard.tsx

// Toggle availability — writes to RTDB instantly
async function toggleAvailability(available: boolean) {
  // RTDB — coordinator sees this in < 1 second
  const location = await getCurrentLocation();
  await set(ref(rtdb, `volunteerPresence/${userId}`), {
    available, location, lastSeen: serverTimestamp()
  });
  // Firestore — persists for records
  await updateDoc(doc(db,'users',userId), { availability: available });
}

// Active mission notification — LIVE
useEffect(() => {
  const q = query(collection(db,'notifications'),
    where('userId','==',userId),
    where('type','==','mission_dispatch'),
    where('read','==',false));
  return onSnapshot(q, snap => {
    setMissionNotification(snap.empty ? null : snap.docs[0].data());
  });
}, [userId]);

// Accept mission
async function acceptMission(notification: Notification) {
  const batch = writeBatch(db);
  // Mark notification read
  batch.update(doc(db,'notifications',notification.id), {read:true});
  // Update mission status
  batch.update(doc(db,'missions',notification.linkedId), {
    status:'dispatched', dispatchedAt:serverTimestamp()
  });
  await batch.commit();
  // Set in RTDB
  await set(ref(rtdb,`missionTracking/${notification.linkedId}`), {
    volunteerId:userId, status:'dispatched',
    location: await getCurrentLocation(), lastUpdate: serverTimestamp()
  });
  navigate(`/volunteer/missions/${notification.linkedId}`);
}
```

6.2 Empathy Engine — Safety Profile + Resources + Brief

```
// src/pages/volunteer/EmpathyEngine.tsx

// All data pre-loaded from mission document — no AI call needed on page load
```

```
// Brief was generated by Gemini at mission dispatch time
const [mission, setMission] = useState(null);

useEffect(() => {
  const unsub = onSnapshot(doc(db,'missions',missionId), snap => {
    setMission(snap.data());
    // Mission document contains:
    // empathyBrief, empathyDecisionTree — Gemini generated at dispatch
    // safetyBriefing — Gemini generated at dispatch from zone.safetyProfile
    // resources[] — pre-assigned by coordinator
  });
  return unsub;
}, [missionId]);

// Fetch zone safety profile for real-time updates
useEffect(() => {
  if (!mission?.zoneId) return;
  getDoc(doc(db,'zones',mission.zoneId)).then(snap => {
    setSafetyProfile(snap.data()?.safetyProfile);
  });
}, [mission?.zoneId]);

// IN-EAR MODE — Gemini Live API (real-time coaching)
// Note: Gemini Live API uses WebSocket — simplified here
async function startInEarMode() {
  setInEarActive(true);
  // Start microphone
  const stream = await navigator.mediaDevices.getUserMedia({audio:true});
  const recorder = new MediaRecorder(stream);
  let chunks = [];

  // Send 10-second audio chunks to Gemini for coaching
  recorder.ondataavailable = async (e) => {
    chunks.push(e.data);
    if (chunks.length >= 5) { // 10 seconds
      const audioBlob = new Blob(chunks, {type:'audio/wav'});
      chunks = [];
      const base64 = await blobToBase64(audioBlob);
      const prompt = `Listen to this conversation clip from a volunteer
        doing ${mission.needType} outreach. Volunteer brief:
${mission.empathyBrief}.
        Suggest ONE short response or next action in ${mission.empathyLanguage}.
        Return JSON: {suggestion:string, confidence:number}`;
      const result = await geminiPro.generateContent([
        prompt, {inlineData:{data:base64,mimeType:'audio/wav'}}]);
      const coaching = JSON.parse(result.response.text());
      setCurrentSuggestion(coaching.suggestion);
    }
  };
  recorder.start(2000); // collect data every 2 seconds
  setMediaRecorder(recorder);
}
```

6.3 My Missions — Active + History

```
// src/pages/volunteer/MyMissions.tsx

// Active mission — LIVE from both Firestore and RTDB
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('assignedTo','==',userId),
    where('status','in',['dispatched','en_route','on_ground']));
  const unsub = onSnapshot(q, snap => {
    const active = snap.empty ? null : snap.docs[0].data();
    setActiveMission(active);
    if (active) {
      // Also listen to RTDB for live status
      onValue(ref(rtdb,`missionTracking/${active.id}`), snap => {
        setLiveStatus(snap.val());
      });
      // Listen to updates subcollection
      onSnapshot(collection(db,'missions',active.id,'updates'), snap => {
        setUpdates(snap.docs.map(d=>d.data()));
      });
    }
  });
  return unsub;
}, [userId]);

// Past missions
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('assignedTo','==',userId),
    where('status','in',['completed','failed']),
    orderBy('completedAt','desc'));
  getDocs(q).then(snap => setPastMissions(snap.docs.map(d=>d.data())));
}, [userId]);

// Close mission
async function closeMission(outcome: string, familiesHelped: number, notes:
string) {
  const batch = writeBatch(db);
  batch.update(doc(db,'missions',activeMission.id), {
    status: outcome==='success' ? 'completed' : 'failed',
    familiesHelped, outcomeNotes: notes,
    completedAt: serverTimestamp()
  });
  // Remove from RTDB
  batch.delete; // can't delete in batch — do separately
  await batch.commit();
  await remove(ref(rtdb, `missionTracking/${activeMission.id}`));

  // Update volunteer stats
  await updateDoc(doc(db,'users',userId), {
    missionsCompleted: increment(1),
    totalHours: increment(calculateHours(activeMission)),
    impactPoints: increment(calculateImpactPoints(activeMission))
  });

  // Add safety signal to zone from this mission
  await updateZoneSafetyProfile(activeMission.zoneId, notes, userId);
```

```
  // Notify coordinator
  await createNotification({
    userId: activeMission.dispatchedBy,
    type:'mission_completed',
    title:`Mission ${activeMission.id} completed`,
    linkedId: activeMission.id
  });
}

// Update zone safety profile from volunteer observations
async function updateZoneSafetyProfile(zoneId: string, notes: string, workerId:
string) {
  const prompt = `Extract safety observations from this volunteer's field note:
'${notes}'.
    Return JSON: {sentiment:'positive'|'neutral'|'negative',
    summary:string (one sentence), safetyFlags:string[]}`;
  const result = await geminiFlash.generateContent(prompt);
  const safety = JSON.parse(result.response.text());

  await updateDoc(doc(db,'zones',zoneId), {
    'safetyProfile.interactions': arrayUnion({
      date: serverTimestamp(), summary: safety.summary,
      sentiment: safety.sentiment, fieldWorkerId: workerId
    }),
    'safetyProfile.specificFlags': arrayUnion(...safety.safetyFlags)
  });

  // Recompute safety score
  const zone = await getDoc(doc(db,'zones',zoneId));
  const interactions = zone.data()?.safetyProfile?.interactions || [];
  const positiveCount = interactions.filter(i=>i.sentiment===='positive').length;
  const newScore = Math.round((positiveCount/interactions.length)*100);
  await updateDoc(doc(db,'zones',zoneId), {
    'safetyProfile.score': newScore,
    'safetyProfile.level': newScore>75?'safe':newScore>50?'moderate':'caution'
  });
}
```

6.4 My Impact — Verified Ledger + DNA + Burnout

```
// src/pages/volunteer/MyImpact.tsx

// User data — LIVE for real-time point updates
const [userData, setUserData] = useState(null);
useEffect(() => {
  return onSnapshot(doc(db,'users',userId), snap => setUserData(snap.data()));
}, [userId]);

// Completed missions with verified impact scores
useEffect(() => {
  const q = query(collection(db,'missions'),
    where('assignedTo','==',userId),
```

```
    where('status','==','completed'),
    where('needScoreAfter','!=',null),
    orderBy('completedAt','desc'));
  getDocs(q).then(snap => setVerifiedMissions(snap.docs.map(d=>d.data())));
}, [userId]);

// Compute impact points + update DNA profile periodically
async function updateDNAProfile() {
  const prompt = `Update volunteer DNA profile based on mission history.
    Completed missions: ${JSON.stringify(verifiedMissions.slice(0,20))}.
    Current profile: ${JSON.stringify(userData.dnaProfile)}.
    Recalculate all 8 dimensions based on actual performance.
    Return JSON: {dnaProfile:{skillMatch,proximity,emotionalCapacity,
    languageMatch,pastSuccess,availability,zoneFamiliarity,burnoutResistance},
    burnoutRisk:'low'|'medium'|'high', burnoutScore:number}`;

  const result = await geminiFlash.generateContent(prompt);
  const updated = JSON.parse(result.response.text());
  await updateDoc(doc(db,'users',userId), {
    dnaProfile: updated.dnaProfile,
    burnoutRisk: updated.burnoutRisk,
    burnoutScore: updated.burnoutScore,
    updatedAt: serverTimestamp()
  });
}

// Verify mission impact after 14-day window
// Run this on component mount for any missions completed 14+ days ago
async function verifyMissionImpact(mission: Mission) {
  const zone = await getDoc(doc(db,'zones',mission.zoneId));
  const currentScore = zone.data()?.currentScore;
  await updateDoc(doc(db,'missions',mission.id), {
    needScoreAfter: currentScore,
    impactScore: mission.needScoreBefore - currentScore
  });
}
```

7. Community Voice Channel — Public Page

```
// src/pages/public/CommunityVoice.tsx — NO AUTH REQUIRED

function generateReferenceNumber(): string {
  return 'NCH-' + Math.floor(1000 + Math.random() * 9000);
}

async function submitCommunityReport(formData: CommunityReportForm) {
  const refNumber = generateReferenceNumber();

  // Detect language and translate to English
  const prompt = `Detect language and translate to English: '${formData.problem}'.
    Return JSON: {language:string, englishTranslation:string}`;
  const langResult = await geminiFlash.generateContent(prompt);
  const langData = JSON.parse(langResult.response.text());

  // Save to communityReports collection
  const reportRef = await addDoc(collection(db,'communityReports'), {
    phone: formData.phone,
    problem: formData.problem,
    problemEnglish: langData.englishTranslation,
    problemType: formData.problemType,
    urgency: formData.urgency,
    location: formData.location,
    referenceNumber: refNumber,
    language: langData.language,
    status: 'received',
    statusUpdates: [{
      status:'received',
      note:'Your report has been received and is being reviewed.',
      timestamp: serverTimestamp()
    }],
    linkedMissionId: null,
    createdAt: serverTimestamp()
  });

  // Link to relevant zone based on location
  const nearestZone = await findNearestZone(formData.location);
  if (nearestZone) {
    await updateDoc(reportRef, { zoneId: nearestZone.id });
    // Add as signal to zone reports for synthesis
    await addDoc(collection(db,'reports'), {
      submittedBy: 'community_voice',
      ngoId: nearestZone.ngoIds[0],
      zoneId: nearestZone.id,
      needType: formData.problemType,
      severity: formData.urgency === 'emergency' ? 'critical' : 'medium',
      familiesAffected: 1,
      inputType: 'community_voice',
      rawData: {communityPhone:formData.phone, referenceNumber:refNumber},
      extractedData: {problem:langData.englishTranslation},
      confidence: 70,
      status: 'synced',
```

```
      createdAt: serverTimestamp()
    });
  }

  setSubmitted(true);
  setReferenceNumber(refNumber);
}

// Status tracking page
async function trackStatus(referenceNumber: string) {
  const q = query(collection(db,'communityReports'),
    where('referenceNumber','==',referenceNumber));
  const snap = await getDocs(q);
  if (snap.empty) { setError('Reference number not found'); return; }
  setTrackedReport(snap.docs[0].data());
}
```

8. Notification System — Global Component

```
// src/components/NotificationPanel.tsx

// Helper to create notifications (called from any page)
export async function createNotification(data: {
  userId: string, type: string, title: string,
  description?: string, linkedId?: string,
  actionable?: boolean, actions?: any[], priority?: string
}) {
  await addDoc(collection(db,'notifications'), {
    ...data, read: false,
    priority: data.priority || 'normal',
    createdAt: serverTimestamp()
  });
}

// In NotificationPanel component — LIVE
useEffect(() => {
  const q = query(collection(db,'notifications'),
    where('userId','==',userId),
    orderBy('createdAt','desc'), limit(50));
  return onSnapshot(q, snap => {
    setNotifications(snap.docs.map(d=>({id:d.id,...d.data()})));
    setUnreadCount(snap.docs.filter(d=>!d.data().read).length);
  });
}, [userId]);

// Mark as read
async function markRead(notificationId: string) {
  await updateDoc(doc(db,'notifications',notificationId), {read:true});
}

async function markAllRead() {
  const unread = notifications.filter(n=>!n.read);
  const batch = writeBatch(db);
  unread.forEach(n => batch.update(doc(db,'notifications',n.id), {read:true}));
  await batch.commit();
}

// FCM Push Notification setup (PWA)
// src/lib/fcm.ts
export async function requestNotificationPermission() {
  const permission = await Notification.requestPermission();
  if (permission === 'granted') {
    const token = await getToken(messaging, {
      vapidKey: import.meta.env.VITE_FCM_VAPID_KEY
    });
    // Save FCM token to user document
    await updateDoc(doc(db,'users',auth.currentUser.uid), { fcmToken: token });
  }
}

// Service worker for background notifications
```

```
// public/firebase-messaging-sw.js
importScripts('https://www.gstatic.com/firebasejs/10.0.0/firebase-app-compat.js');
importScripts('https://www.gstatic.com/firebasejs/10.0.0/firebase-messaging-
compat.js');
firebase.initializeApp({ /* same config */ });
const messaging = firebase.messaging();
messaging.onBackgroundMessage(payload => {
  self.registration.showNotification(payload.notification.title, {
    body: payload.notification.body, icon: '/nexus-icon.png'
  });
});
```

9. Automated AI Jobs — Background Synthesis & Pre-seeding

Since we are NOT using Cloud Functions (keys in .env), these jobs run client-side when coordinator opens the dashboard. A simple scheduler checks if the job ran recently before executing.

```
// src/lib/autoJobs.ts
// Called when coordinator dashboard mounts

export async function runAutoJobsIfNeeded(ngoId: string) {
  const metaDoc = await getDoc(doc(db,'_jobMeta',ngoId));
  const meta = metaDoc.data() || {};
  const now = Date.now();

  // Synthesis: run if last run > 15 minutes ago
  if (!meta.lastSynthesis || now - meta.lastSynthesis > 15*60*1000) {
    await runSynthesisJob(ngoId);
    await setDoc(doc(db,'_jobMeta',ngoId), {lastSynthesis:now}, {merge:true});
  }

  // Forecast: run if last run > 7 days ago
  if (!meta.lastForecast || now - meta.lastForecast > 7*24*60*60*1000) {
    await runForecastJob(ngoId);
    await setDoc(doc(db,'_jobMeta',ngoId), {lastForecast:now}, {merge:true});
  }

  // Burnout: run if last run > 7 days ago
  if (!meta.lastBurnout || now - meta.lastBurnout > 7*24*60*60*1000) {
    await runBurnoutJob(ngoId);
    await setDoc(doc(db,'_jobMeta',ngoId), {lastBurnout:now}, {merge:true});
  }
}

// Pre-seed city with public data when NGO onboards
async function preSeedCityData(ngoId: string, city: string, zones: string[]) {
  const prompt = `Generate baseline community data for ${city}, India.
    Create realistic zone health scores and signal counts based on
    typical urban Indian neighborhood patterns for zones: ${zones.join(', ')}.
    Return JSON: zones:[{zoneId, currentScore:number, riskLevel, signalCounts:{
    food,education,health,substance,shelter}, safetyProfile:{score,level}}]`;

  const result = await geminiFlash.generateContent(prompt);
  const seedData = JSON.parse(result.response.text());

  const batch = writeBatch(db);
  for (const zone of seedData.zones) {
    const zoneRef = doc(db,'zones',zone.zoneId);
    batch.set(zoneRef, {
      ...zone, ngoIds:[ngoId], city,
      scoreHistory:[{week:'W01',score:zone.currentScore,actual:false}],
      activeMissions:0, updatedAt:serverTimestamp()
    }, {merge:true});
  }
```

```
  await batch.commit();
}
```

10. Final Sidebar Structure + React Router Setup

10.1 Updated Coordinator Sidebar

Section	Nav Items
OVERVIEW	Dashboard · Analytics
OPERATIONS	Need Terrain Map · Alerts Feed · Volunteers · Missions · Resources (NEW)
INTELLIGENCE	Intelligence Hub (merged — Gemini Insights + Forecast + Trust & Impact + Living Constitution + Echo)
SETTINGS	Organisation · Team
REMOVED	Impact Reports (merged) · Integrations (removed)

10.2 React Router Configuration

```
// src/App.tsx — Route definitions
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';

// Public routes
<Route path='/' element={<LandingPage />} />
<Route path='/login' element={<LoginPage />} />
<Route path='/signup' element={<SignupPage />} />
<Route path='/community-voice' element={<CommunityVoice />} />
<Route path='/community-voice/track' element={<TrackReport />} />

// Coordinator routes (protected)
<Route path='/coordinator' element={<ProtectedRoute role='coordinator' />}>
  <Route path='dashboard' element={<CoordinatorDashboard />} />
  <Route path='map' element={<NeedTerrainMap />} />
  <Route path='alerts' element={<AlertsFeed />} />
  <Route path='volunteers' element={<VolunteerManagement />} />
  <Route path='missions' element={<Missions />} />
  <Route path='resources' element={<ResourceInventory />} />
  <Route path='intelligence' element={<IntelligenceHub />} />
  <Route path='organisation' element={<Organisation />} />
  <Route path='team' element={<Team />} />
</Route>

// Field worker routes (protected)
<Route path='/fieldworker' element={<ProtectedRoute role='fieldworker' />}>
  <Route path='dashboard' element={<FieldWorkerDashboard />} />
  <Route path='scan' element={<ScanSurvey />} />
  <Route path='voice' element={<VoiceReport />} />
  <Route path='reports' element={<MyReports />} />
  <Route path='mission' element={<ActiveMission />} />
  <Route path='profile' element={<FieldWorkerProfile />} />
</Route>

// Volunteer routes (protected)
```

```
<Route path='/volunteer' element={<ProtectedRoute role='volunteer' />}>
  <Route path='dashboard' element={<VolunteerDashboard />} />
  <Route path='missions' element={<MyMissions />} />
  <Route path='missions/:missionId' element={<ActiveMissionDetail />} />
  <Route path='empathy/:missionId' element={<EmpathyEngine />} />
  <Route path='impact' element={<MyImpact />} />
  <Route path='profile' element={<VolunteerProfile />} />
</Route>

// Protected Route component
function ProtectedRoute({ role }: { role: string }) {
  const { user, loading } = useAuthStore();
  if (loading) return <LoadingScreen />;
  if (!user) return <Navigate to='/login' />;
  if (user.role !== role) return <Navigate to='/login' />;
  return <Outlet />;
}
```

11. Final Summary — Complete Feature Checklist

Feature	Page	AI Used	Real-time
User signup + role routing	Login / Signup	Gemini (city pre-seed)	No
Need terrain map with live heat	Need Terrain Map	None (pre-computed)	Yes — onSnapshot zones
Gemini insight synthesis	Intelligence Hub Tab 1	Gemini Pro (synthesis)	Yes — onSnapshot insights
Need forecasting 4 weeks	Intelligence Hub Tab 2	Gemini Pro (predict)	No — job-based
Trust score + impact ledger	Intelligence Hub Tab 3	Gemini Flash (score)	Partial — score onSnapshot
Donor PDF report	Intelligence Hub Tab 3	Gemini Pro (narrative)	No — on demand
Government policy brief	Intelligence Hub Tab 4	Gemini Pro (draft)	Partial — status onSnapshot
Community Echo broadcast	Intelligence Hub Tab 5	Gemini Flash (write)	Partial — status onSnapshot
Drift alerts auto-generated	Alerts Feed	Gemini (via synthesis)	Yes — onSnapshot alerts
Volunteer DNA matching	Volunteer Mgmt + Create Mission	Gemini Pro (score)	On demand
AI scoring breakdown visible	Volunteer Mgmt	Pre-computed from DNA	No
Duplicate mission detection	Create Mission + Scan Survey	None — Firestore query	No
Resource inventory management	Resource Inventory	None	Yes — onSnapshot
Resources shown at dispatch	Create Mission Step 3	None — Firestore query	No
Mission create + dispatch	Missions Page	Gemini (brief + safety)	Yes — onSnapshot missions
Live mission tracking	Missions + Active Mission	None	Yes — RTDB sub-second
Paper survey OCR	Scan Survey	Gemini Vision	No — on submit
Voice report transcription	Voice Report	Gemini Audio	No — on stop
Report merging into missions	Scan + Voice + Cloud	None — Firestore logic	No
Zone safety profile update	My Missions (close)	Gemini Flash (extract)	No — on close
Field worker survey missions	Active Mission (FW)	None	Yes — RTDB status

Empathy Engine full brief	Empathy Engine	Pre-computed at dispatch	No
Safety profile in brief	Empathy Engine	Pre-computed at dispatch	No
Resources in brief	Empathy Engine	None — Firestore data	No
In-ear mode coaching	Empathy Engine	Gemini Pro (live audio)	Yes — streaming
My Missions + close outcome	My Missions	Gemini Flash (safety)	Yes — onSnapshot
Volunteer DNA profile update	My Impact	Gemini Flash (update)	No — on trigger
Burnout prediction	My Impact + Dashboard	Gemini Flash (compute)	No — weekly job
Impact verification 14-day	My Impact	None — score delta	No — on load
Community voice submit	Community Voice	Gemini Flash (translate)	No
Community report tracking	Community Voice Track	None — Firestore query	No
Push notifications	All roles	None	Yes — FCM
Notification panel live	All roles	None	Yes — onSnapshot
Availability toggle	Volunteer Dashboard	None	Yes — RTDB instant
Inter-NGO collaboration card	Organisation	Gemini (weekly suggest)	No

FINAL IMPLEMENTATION NOTE: This document contains every API call, Firestore query, real-time listener, AI prompt, and data flow in the NEXUS project. All Gemini API calls use import.meta.env.VITE_GEMINI_API_KEY from the .env file. All Firebase calls use the initialized instances from src/lib/firebase.ts. Service-account.json is only referenced by GOOGLE_APPLICATION_CREDENTIALS for local scripts — never imported in React code. Copy each code block into the corresponding file path shown. Every feature in this document maps to a visible UI component in the NEXUS frontend.